AI Hackathon with Nebius & NVIDIA

Challenge proposal: a dark factory for bug resolution

One-liner

Build the confidence layer that lets a repo fix its own trivial bugs and ship them to production with no human approving the change, and have people actually trust it.

The problem

We can already point a coding agent at a bug and get a plausible fix. That is not the hard part anymore. The hard part is that nobody wants to merge and deploy that fix without a human reading it first. So the human stays in the loop, the loop becomes the bottleneck, and the “autonomous” part quietly disappears.

That makes this a trust problem, not a capability one. The question the challenge asks is this: what has to be true before a team lets a fix ship unattended and still sleeps fine that night?

The frame: dark factory

In manufacturing, a dark factory runs with the lights off because no human needs to be on the floor. What makes that safe is not clever robots. It is the jigs, sensors, interlocks, and automatic line-stops around them. Trust lives in the system, not in any single machine.

This challenge has the same shape. The bug-fixing agent is the robot arm, and nobody cares how smart the arm is. They care about the interlocks. So that is what teams build: not the arm, but everything around it. The arm stays a black box we plug in and never look inside.

What teams build

The deliverable is that confidence layer, and it runs as a pipeline: pre-checks, then the black box, then post-checks, then a gated deploy, with an audit trail across the whole thing.

Pre-checks decide whether the box runs at all. Is the bug in scope, and can we automatically generate a test that reproduces it? If not, it goes straight to a human and the box never starts.

Then the black box runs. It takes a qualified bug and a failing test, and hands back a diff. The confidence layer does not touch what happens inside and does not host its environment.

Post-checks decide whether that diff is allowed to ship, and there are two kinds. The oracle checks the behavior: the reproducing test goes from red to green and the full regression suite stays green, which is the evidence that the bug is gone and nothing else broke. Diff inspection checks the change itself: it stays small and localized, touches no protected paths, adds no secrets, and changes no dependencies. This is the security side, done by reading the output rather than caging the runtime. If either check fails, the fix is rejected and the bug is escalated.

What passes goes out through a gated deploy, behind a flag and a canary, with automatic rollback when an SLO slips. The worst case stays small and undoes itself.

The audit trail records every step: the bug, the qualification decision, the diff, the verifying evidence, and the deploy result, immutable and signed. The human is out of the loop while this runs and fully on the loop afterward, able to pull the whole record on demand and replay any decision.

Under all of it sits one default: when a check does not clearly pass, the system hands the bug to a person instead of guessing. That default is what makes the line safe to run unattended.

Scope

Two things keep this doable in a day.

First, bug selection. The system only auto-resolves trivial bugs, and trivial has a precise definition: a test can be generated to reproduce the bug automatically, and the fix is a small, localized diff. Everything else gets escalated. Scoping to trivial bugs is not a cop-out, because the gate that enforces that definition is exactly what earns the trust.

Second, the fixer itself is out of scope. Teams treat it as a black box behind a clean interface and can drop in anything, from a one-line script to a full multi-agent factory. None of the confidence is meant to come from that box. It comes from what surrounds it: the pre-checks, the oracle and diff inspection, the gated deploy, and the audit trail.

Where the human sits

Out of the loop for the action, on the loop for governance. No real-time approval on the merge or deploy, with a full audit available on demand and the right to step in, pause the line, or roll back at any time.

The demo that proves it

Seed a repo with a handful of trivial bugs and one genuinely nasty one. The system resolves the trivial bugs end to end, from bug report to verified fix to deployed change to audit entry, while people watch and touch nothing. Then it hits the nasty bug, refuses it, and escalates to a human.

Why it fits

Every engineering org at SAP carries a long tail of small bugs that are individually cheap and collectively expensive. This goes straight at that tail. And the trust problem behind it is bigger than bug-fixing: if the confidence layer holds, the same interlocks extend to other autonomous changes later. The escalation gate and the audit trail are reusable well beyond this one use case.

It is also doable in a day. Teams do not build the whole factory. They build the pre-checks and post-checks around an off-the-shelf coding agent, on one repo, with a seeded bug set. The compute from Nebius and NVIDIA makes the multi-agent and verification passes practical to run live during the event.

Future improvements

A runtime sandbox for the agent. The pipeline validates the diff after the box runs, which catches a wrong or out-of-bounds fix but not a rogue runtime: an agent that reaches the network or reads a secret while executing would slip past. Real containment, with scoped file access, no secret

access, and a network allowlist, closes that gap. It is out of scope for a one-day build, because hosting the environment for an arbitrary plugged-in agent is a lot of work for little demo value, but it is the obvious next step for anything running beyond the hackathon.